Jetpack Compose

Jetpack Compose

- Jetpack Compose is a modern **toolkit** for building native Android UI. Jetpack Compose simplifies and accelerates UI development on Android with less code.
- The code you're writing to design the UI is written only in Kotlin, rather than having it split between Kotlin and XML.
- It's much easier to trace through code when it's all written in the same language and often the same file, rather than jumping back and forth between Kotlin and XML"

Define a composable function

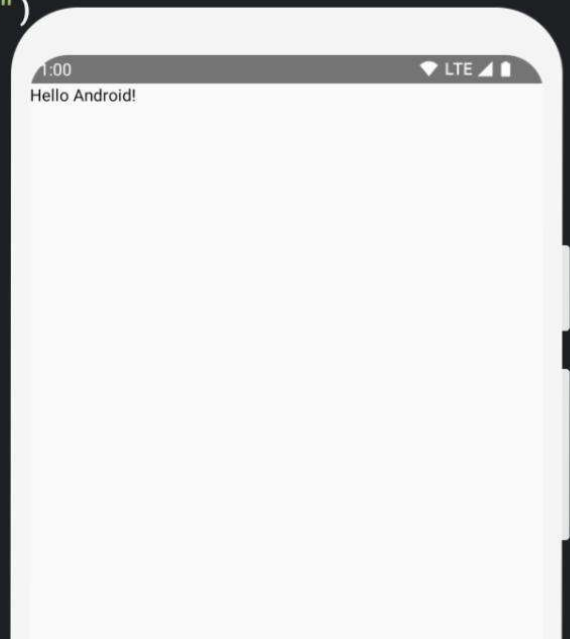
To make a function composable, add the `@Composable` annotation. To try this out, define a `MessageCard` function which is passed a name, and uses it to configure the text element.

HIDE PREVIEW

```
// ...
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MessageCard("Android")
        }
    }
}

@Composable
fun MessageCard(name: String) {
    Text(text = "Hello $name!")
}
```



Preview your function in Android Studio

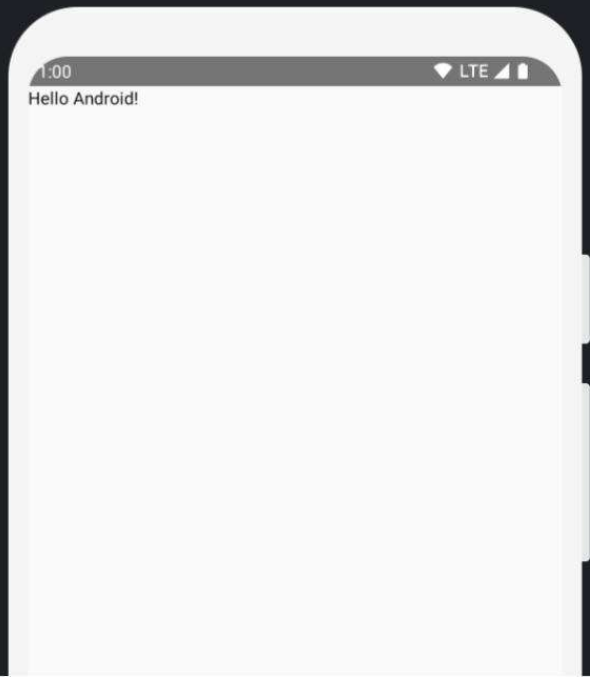
The `@Preview` annotation lets you preview your composable functions within Android Studio without having to build and install the app to an Android device or emulator. The annotation must be used on a composable function that does not take in parameters. For this reason, you can't preview the `MessageCard` function directly. Instead, make a second function named `PreviewMessageCard`, which calls `MessageCard` with an appropriate parameter. Add the `@Preview` annotation before `@Composable`.

HIDE PREVIEW

```
// ...
import androidx.compose.ui.tooling.preview.Preview

@Composable
fun MessageCard(name: String) {
    Text(text = "Hello $name!")
}

@Preview
@Composable
fun PreviewMessageCard() {
    MessageCard("Android")
}
```



UI ELEMENTS

UI (User Interface) refers to the visual and interactive elements of a software application that **users interact with**. It includes **buttons, text fields, images, menus**, and other components that enable users to navigate and use an application.

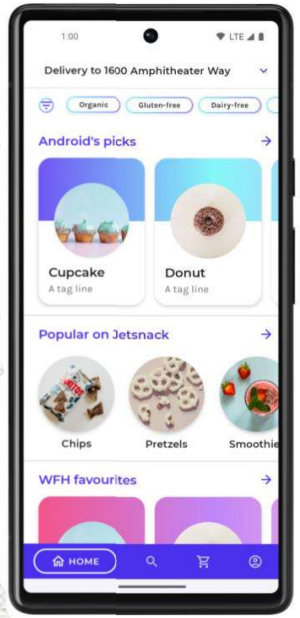
3/12/2026

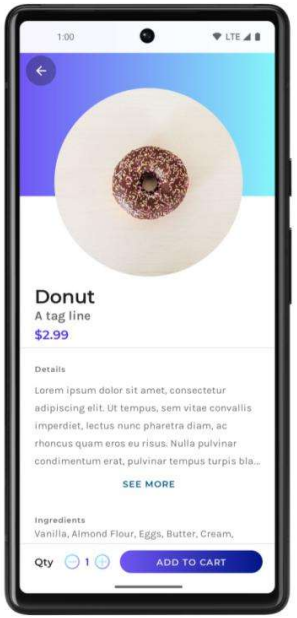
UI ELEMENTS

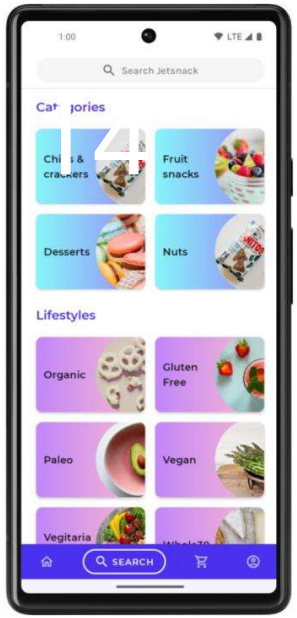
UI Elements are small individual components that make up an application's interface.

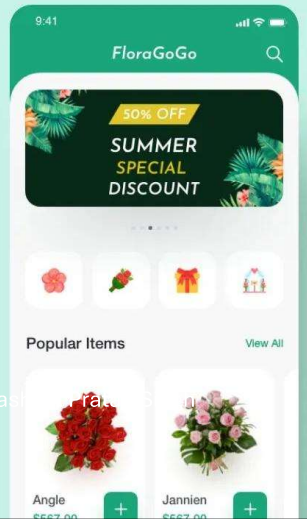
These elements are defined in layouts and controlled by activities.

Layouts define UI structure whereas activities manage UI behaviour.



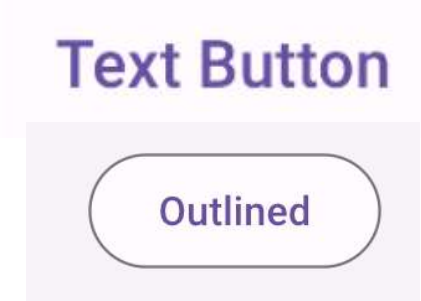






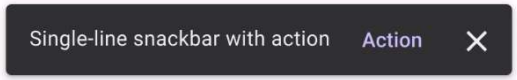
UI Elements in Jetpack Compose

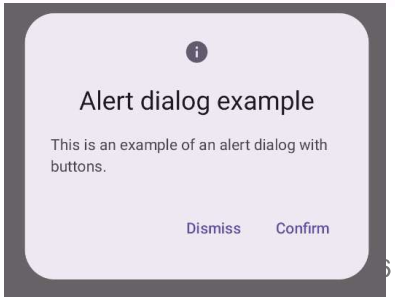
UI ELEMENT	FUNCTION	DESCRIPTION
Text	Text()	Displays text on the screen.
Button	Button()	A Clickable Button with text or icon
Outlined Button	OutlinedButton()	A Button with outlined border
Text Button	TextButton()	A Button without background but with a text
Icon Button	IconButton()	A Button for icons only



UI Elements in Jetpack Compose

UI ELEMENT	FUNCTION	DESCRIPTION
Text Field	TextField()	Editable text input field.
Outlined text field	OutlinedTextField()	An input field with outlined border.
Image	Image()	To Display Image
Snack bar	SnackBar()	Displays a temporary message
Alert dialog	AlertDialog()	Shows a dialog with buttons.





UI Elements: Text()

Text() is used to display some text on the app screen.

```
@Composable
fun SimpleText() {
    Text("Hello World")
}
```

Hello World

UI Elements: Text()

Text() has various parameters to customise it’s appearance and behavior.

1. text : String - The text to be displayed.
2. color : Color – Sets the color of the text
3. fontSize : TextUnit – Sets size of the text
4. fontStyle: FontStyle – Allows setting different font styles
5. fontFamily: FontFamily – Specifies a font family.

UI Elements: Text(“Hello”, color = Color.Red)

BasicColors

1. Color.White
2. Color.Gray
3. Color.Red
4. Color.Green
5. Color.Blue
6. Color.Yellow
7. Color.Cyan
8. Color.Magenta
9. Color.LightGray
10. Color.DarkGray
11. Color.Transparent (fully transparent)
12. Color.Unspecified (no color applied)

UI Elements: Text(“Hello”, color = Color.Red)

Code Example

```
@Composable
fun ColoredText() {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(text = "Red Text", color = Color.Red, fontSize = 20.sp)
        Text(text = "Green Text", color = Color.Green, fontSize = 20.sp)
        Text(text = "Blue Text", color = Color.Blue, fontSize = 20.sp)
        Text(text = "Custom Color", color = Color(0xFFFFA500), fontSize =
20.sp) // Orange
    }
}
```

MainActivity.kt

Code Split Design

Up-to-date

```
1  package com.example.composetutorial
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.compose.material3.Text
7  import androidx.compose.runtime.Composable
8  import androidx.compose.ui.tooling.preview.Preview
9
10 class MainActivity : ComponentActivity() {
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContent {
14             MessageCard( name: "Android")
15         }
16     }
17 }
18
19 @Composable
20 fun MessageCard(name: String) {
21     Text(text = "Hello $name!")
22 }
23
24 @Preview
25 @Composable
26 fun PreviewMessageCard() {
27     MessageCard( name: "Android")
28 }
29
30
31
32
33
```

PreviewMessageCard

Hello Android!

```
@composable
fun end of Slide(){
            Text(text  =  “THANK YOU”,
color = Color.Green)
}
```